

Entwurf digitaler Systeme

4. Minimierung Boolescher Funktionen

4. Minimierung Boolescher Funktionen

- Es ist sichtbar geworden, dass mit Hilfe **verschiedener Regeln** Funktionen vereinfacht werden konnten.
- Durch diese Vereinfachungsschritte ist es gelungen, eine erhebliche **Minimierung des Schaltungsaufwands** zu erreichen.
- Allerdings muss auch festgestellt werden, dass die bisherige Minimierung von Booleschen Funktion im Wesentlichen auf der Methode des „**scharfen Hinsehens**“ und der intuitiv richtigen Anwendung (Reihenfolge, passende Regeln, usw.) beruhte.
- Unter Berücksichtigung eines möglichst optimalen – **kostenminimalen** – Schaltungsentwurfs, ist diese Vorgehensweise nicht akzeptabel, da es keinen Überblick darüber gibt, wann eine optimale Schaltung gefunden wurde.

4. Minimierung Boolescher Funktionen

- Die Optimierung logischer Schaltungen stellt eines der komplexesten Gebiete der Digitaltechnik dar. Es haben sich deshalb verschiedene Minimierungsverfahren herausgebildet. Die Minimierung Boolescher Funktionen verfolgt immer das Ziel, alle in einer Funktion redundanten Terme zu eliminieren, um die Schaltung kostenminimal zu realisieren. In den meisten Fällen werden aber praktische Lösungen angestrebt, die nicht immer das Aufwandsminimum darstellen – dafür aber operabel sind.

4. Minimierung Boolescher Funktionen

■ Definition 4.1 (Kostenminimum)

- Der Kostenfaktor F_K ist definiert als die Summe der Anzahl N_E aller in einer kombinatorischen Schaltung auftretenden Literale (Eingänge) und die Anzahl N_G der verwendeten Gatter. Dabei werden Negationen und deren Eingänge **nicht** mitgezählt.

$$F_K := N_E + N_G$$

- Stets soll für die Kostenfunktion F_K gelten:

$$F_K \stackrel{!}{=} \text{Minimum}$$

4. Minimierung Boolescher Funktionen

- Es ist das Ziel Boolesche Funktionen mit dem minimalen operablen Kostenaufwand zu erzeugen. Dieses geschieht wiederum in der disjunktiven oder konjunktiven Form. In diesem Zusammenhang spricht man dann von einer **disjunktiven Minimalform (DMF)** oder **konjunktiven Minimalform (KMF)**.
- Besondere Bedeutung haben diese Formen für **zweistufige kombinatorische Schaltungen** erhalten, die im nächsten Kapitel applikationsbezogen näher betrachtet werden. Zunächst sollen einige Verfahren prinzipiell vorgestellt werden.

4. Minimierung Boolescher Funktionen

- **Minimierung mit Hilfe der Booleschen Algebra**
 - Dieses Verfahren haben wir bereits in den vorherigen Kapiteln kennengelernt und implizit benutzt, um den Umgang mit den Regeln der Booleschen Algebra zu festigen. Es eignet sich in der Regel nur für bis zu drei Variablen, um eine sinnvolle Lösung nach Definition 4.1 zu gewinnen.
 - Wie bereits angedeutet, ist für dieses Verfahren **viel Übung und Intuition** notwendig. Für mehr als vier Variablen ist dieses Vorgehen kaum sinnvoll, da sehr schnell der Überblick verloren geht. Dennoch erweist sich der Umgang mit den Regeln der Booleschen Algebra als unbedingt notwendig, da üblicherweise zumindest gewisse Umformungen vor dem Minimierungsschritt unabdingbar sind.

4. Minimierung Boolescher Funktionen

■ Graphische Verfahren

- Die bekanntesten Verfahren sind das **Venn-Diagramm** und das **Karnaugh-Veitch-Diagramm**. Das Venn-Diagramm stellt mit Hilfe der Darstellungsformen der Mengenalgebra (Vereinigungsmenge: ODER, Schnittmenge: UND) einfache Zusammenhänge der Booleschen Algebra dar. Dieses Verfahren ist nicht für hochzahlige Variablenminimierungen geeignet.
- Eine große Bedeutung hat das unabhängig von Karnaugh und Veitch entwickelte graphische Verfahren erlangt. Dieses wird auch als **Karnaugh-Plan** oder kurz **K-Plan (KP)** bzw. **K-Map (KM)** bezeichnet. Wir wollen uns dieser Bezeichnungsweise anschließen, da sie **international üblich** ist. Der K-Plan eignet sich für die Minimierung von bis zu sechs Eingangsvariablen, danach wird er unübersichtlich. Er liefert mit etwas Übung – recht anschaulich – gute Ergebnisse. Deshalb wird das Verfahren zukünftig unter anderem verwendet.

4. Minimierung Boolescher Funktionen

■ Algorithmische Verfahren

- Algorithmische Verfahren basieren auf einer formalen Reduktion der Eingangsvariablen. Sie sind deshalb zur Implementierung auf Computern geeignet. Das bekannteste formale Verfahren ist das nach **Quine-McCluskey (QMC)** benannte. Es können beliebig viele Eingangsvariable verwendet werden. Es scheitert nur dann, wenn der Ausführende die Übersicht verliert. Das QMC-Verfahren ist in einer Anzahl von Softwarewerkzeugen implementiert.
- Neben diesem gibt es heute weitere, die zur Implementierung geeignet sind. Es handelt sich um so genannte **heuristische Verfahren**, die mit iterativen Methoden das Kostenminimum erreichen. Diese haben aufgrund ihrer Leistungsfähigkeit eine weite Verbreitung gefunden, da sie die Leistungsfähigkeit heutiger Personalcomputer und Workstations ausnutzen. Zu nennen sind das **Presto-** und **Espresso-**Verfahren. Beide wurden an der amerikanischen **Berkeley-University** entwickelt.

4. Minimierung Boolescher Funktionen

■ Grundsätzlicher Ansatz aller Minimierungsverfahren

- Für die DNF:

$$AB + \overline{A}\overline{B} = A.$$

- Für die KNF :

$$(A + B) \cdot (A + \overline{B}) = A.$$

$$Z = K_0 + K_1 + \dots + K_i + K_j + \dots + K_{M-1}.$$

$$\dots + \underbrace{K_i}_{\text{blue circle}} + \underbrace{K_j}_{\text{red circle}} + \dots$$

$$\ddot{x}_i = \begin{cases} \text{entweder} & x_i \\ \text{oder} & \overline{x_i} \end{cases}$$

$$= \dots + (\ddot{x}_0 \cdot \ddot{x}_1 \cdot \dots \cdot \ddot{x}_{k-1} \cdot x_k \cdot \ddot{x}_{k+1} \cdot \dots \cdot \ddot{x}_{N-1}) + \dots$$

$$\dots + (\ddot{x}_0 \cdot \ddot{x}_1 \cdot \dots \cdot \ddot{x}_{k-1} \cdot \overline{x_k} \cdot \ddot{x}_{k+1} \cdot \dots \cdot \ddot{x}_{N-1}) + \dots = \dots$$

Nur gültig mit dem gesprochenen Wort

Prof. Dr.-Ing. Volker Lohweg, Roland Hildebrand

4. Minimierung Boolescher Funktionen

$$= \dots + (\ddot{x}_0 \cdot \ddot{x}_1 \cdot \dots \cdot \ddot{x}_{k-1} \cdot \ddot{x}_{k+1} \cdot \dots \cdot \ddot{x}_{N-1}) + \dots$$

- Beachten Sie, dass nun aus den beiden Konjunktionen *eine* neue, um *eine* Variable reduzierte Konjunktion entstanden ist. Dieses Vorgehen ist in allen Minimierungsverfahren zu finden.

4.1 Graphische Verfahren

- Bei den graphischen Verfahren betrachten wir ausschließlich den K-Plan für bis zu vier Variablen.
- Mit dem K-Plan lassen sich sowohl disjunktive als auch konjunktive Minimalformen realisieren.
- In den meisten Fällen wird von der disjunktiven Minimalform Gebrauch gemacht, weil sie für zweistufige Logikschaltungen in *applikationsspezifischen Schaltungen* (*Programmable Logic Device, PLD*) hervorragend einsetzbar ist.
- Da Ihnen die Umsetzung von disjunktiven Schaltungen auf konjunktive nunmehr geläufig ist, werden wir die konjunktive Minimalform nur am Rande betrachten.

4.1.1 Karnaugh-Plan

- Der K-Plan stellt die Anordnung von Mintermen (Maxterme) graphisch dar.
- In einem Diagramm werden alle Minterme (Maxterme) so angeordnet, dass diejenigen Terme, die sich nur in einer Eingangsvariablen unterscheiden, direkt nebeneinander liegen.
- Der K-Plan für eine Funktion mit N Variablen besteht aus einem zweidimensionalen Feld mit 2^N Plätzen. Jedem Platz ist genau eine mögliche Konjunktion oder Disjunktion zugeordnet.
- Beispiel I:

Umordnung
(0, 1, 2, 3 $\rightarrow$ 0, 1, 3, 2)

i	B	A	Z
0	0	0	1
1	0	1	1
3	1	1	0
2	1	0	0

$$Z = \overline{B} \cdot \overline{A} + \overline{B} \cdot A.$$

4.1.1 Karnaugh-Plan

- Der entsprechende K-Plan besitzt folgendes Aussehen: Seitlich werden die Variablennamen aufgetragen, innerhalb der Plätze werden – im Fall der **KDNF oder DNF** – die Ausgangswerte der Booleschen Funktion $Z = 1$ eingetragen. Im Fall der **KKNF oder KNF** wird $Z = 0$ aufgetragen.

	$\overline{A}$	A
$\overline{B}$	1 $i = 0$	1 $i = 1$
B	0 $i = 2$	0 $i = 3$

4.1.1 Karnaugh-Plan

- Es ist sofort erkennbar, dass alle Kombinationen mit einer Änderung nur einer Variablen nebeneinander liegen ((0, 1), (0, 2), (1, 3), (2, 3)).
- Es ist weiterhin erkennbar, dass Z ausschließlich von $\overline{B}$ abhängig ist. Man fasst deshalb die beiden Einsen mit einer Kennzeichnung zusammen, um die Abhängigkeit klar zu machen.

	$\overline{A}$	A	
$\overline{B}$	1 $i = 0$	1 $i = 1$	$Z = \overline{B}.$
B	0 $i = 2$	0 $i = 3$	

Vergleichen Sie!

$$\begin{aligned}
 Z &= \overline{B} \cdot \overline{A} + \overline{B} \cdot A = \dots \\
 \dots &= \overline{B} \cdot (\overline{A} + A) = \overline{B} \cdot 1 = \overline{B}.
 \end{aligned}$$

4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Minimierungsvorschriften nach Karnaugh
 - Nachdem die zu minimierende Boolesche Funktion mit ihren Ausgangsbelegungen $f_i \in \{0, 1, \times\}$ in den K-Plan eingetragen ist, erfolgt das Zusammenfassen von Belegungsplätzen zu Blöcken. Dabei muss zunächst entschieden werden, ob eine konjunktive oder disjunktive Minimalform generiert werden soll. Im Falle der DMF werden Einsen zusammengefasst, während für die KMF entsprechend Nullen in Blöcke aufgenommen werden.
 - Es gilt:
 - Ein Zweierblock eliminiert eine Variable.
 - Ein Viererblock eliminiert zwei Variablen.
 - Ein Achterblock eliminiert drei Variablen.
 - USW.

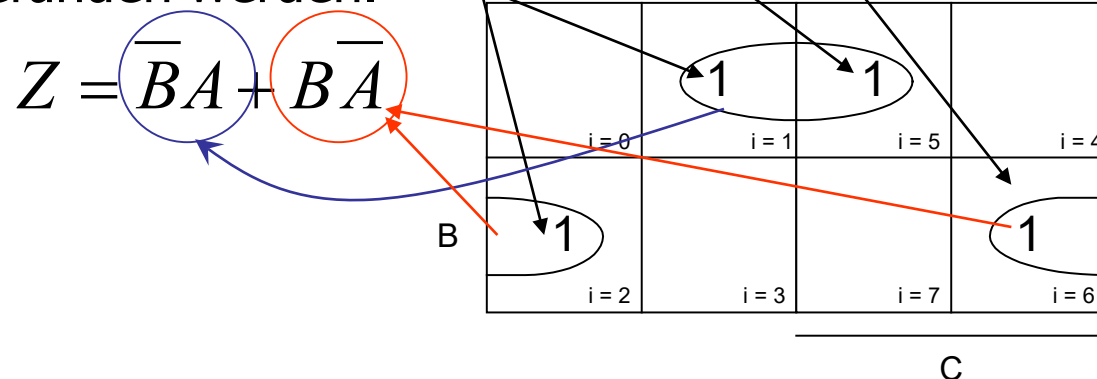
4.1.1.1 Minimierungsvorschriften nach Karnaugh

Es ist Ziel, eine (möglichst) minimale Anzahl von Blöcken bei einer maximalen Blockgröße zu bilden. Redundante oder doppelt belegte Terme sind zu vermeiden. Ein Variablenplatz kann Bestandteil mehrerer Blöcke sein.

Beispiel II: Gegeben sei folgende Gleichung:

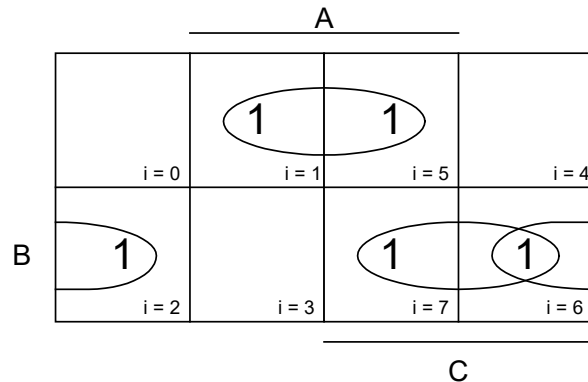
$$Z = \overline{C}\overline{B}A + \overline{C}B\overline{A} + C\overline{B}A + CB\overline{A}$$

Es soll die DMF gefunden werden.



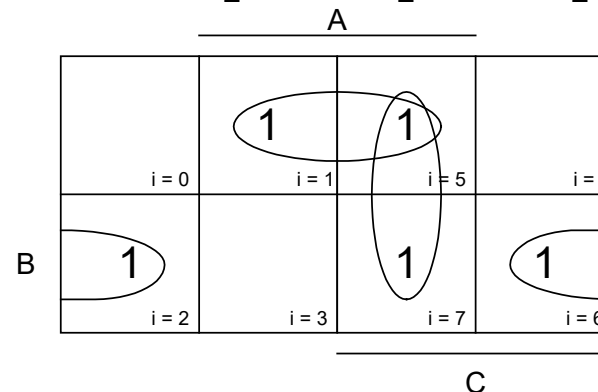
4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel III
 - Es soll die DMF gefunden werden. Man erhält in diesem Beispiel **zwei** äquivalente Lösungen.



$$Z_1 = CB + \bar{B}A + B\bar{A}$$

C	B	A	Z
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

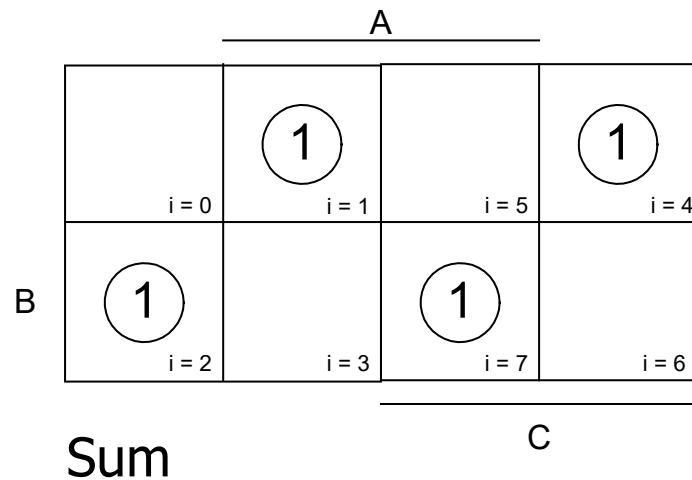


$$Z_2 = CA + \bar{B}A + B\bar{A}$$

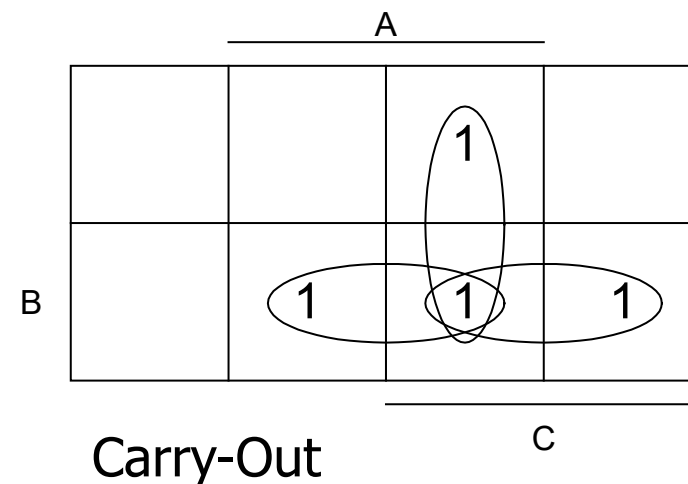
4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel IV
 - Volladdierer

$$S = \bar{A} \cdot \bar{B} \cdot C + A \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot \bar{C} + A \cdot B \cdot C$$

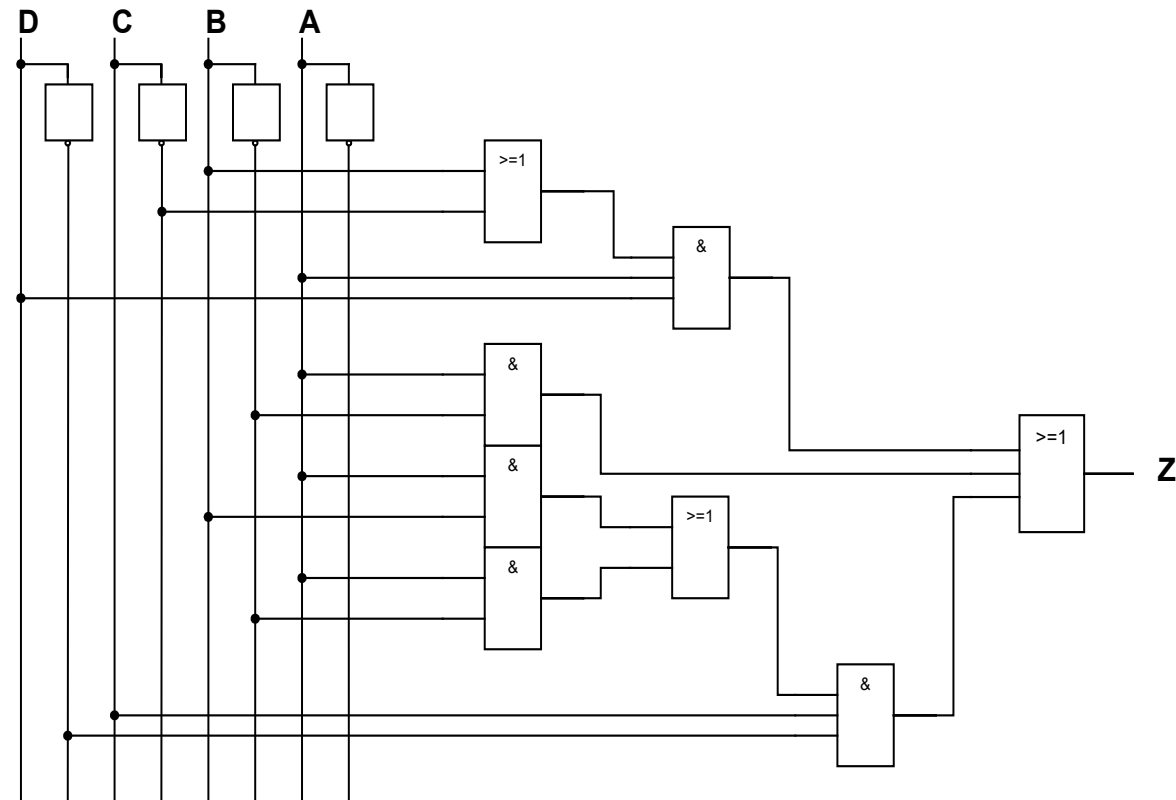


$$O = A \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$$



4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel V
 - Gleichung mit 4 Variablen



$$Z = f(A, B, C, D) = A \cdot D \cdot (B + \overline{C}) + A \cdot \overline{B} + (AB + \overline{B}A) \cdot C \cdot \overline{D}$$

$$F_K = N_E + N_G = 19 + 8 = 27$$

4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel V
 - Teil 2

$$Z = f(A, B, C, D) = A \cdot D \cdot (B + \bar{C}) + A \cdot \bar{B} + (AB + \bar{B}A) \cdot C \cdot \bar{D}$$

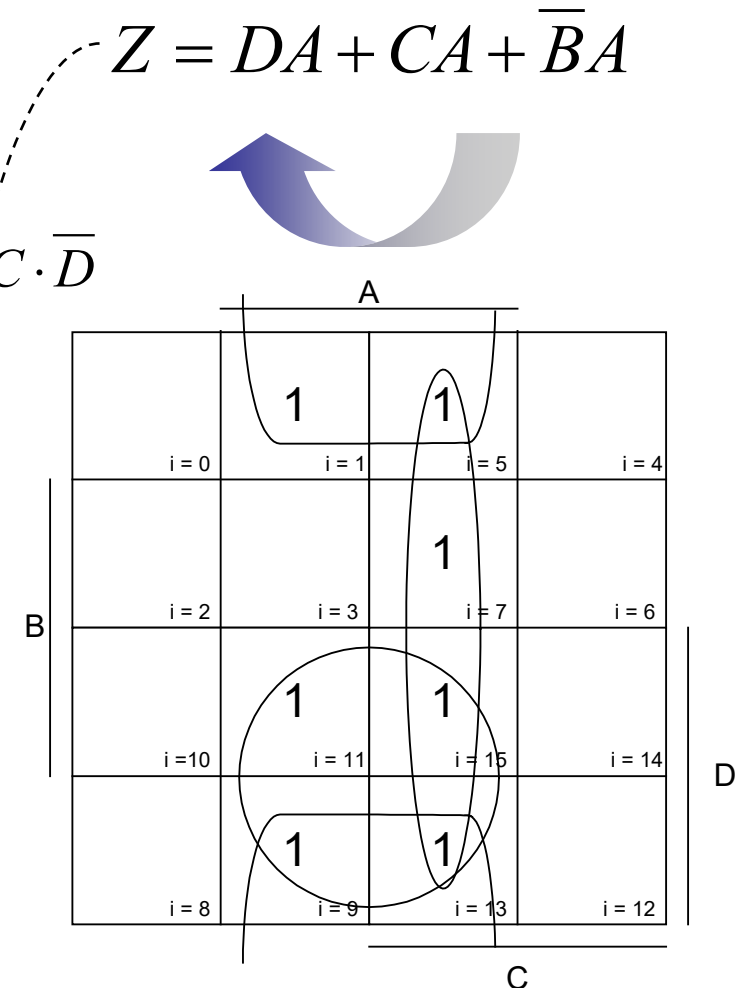
$$Z = DCBA + \bar{D}\bar{C}\bar{B}A + \bar{D}\bar{C}\bar{B}A + DC\bar{B}A + \dots$$

$$\dots + \bar{D}\bar{C}\bar{B}A + \bar{D}\bar{C}\bar{B}A + \bar{D}\bar{C}\bar{B}A.$$

Kostenrechnung

$$F_K = N_E + N_G = 19 + 8 = 27$$

$$\min(F_K) = \min(N_E + N_G) = 9 + 4 = 13$$

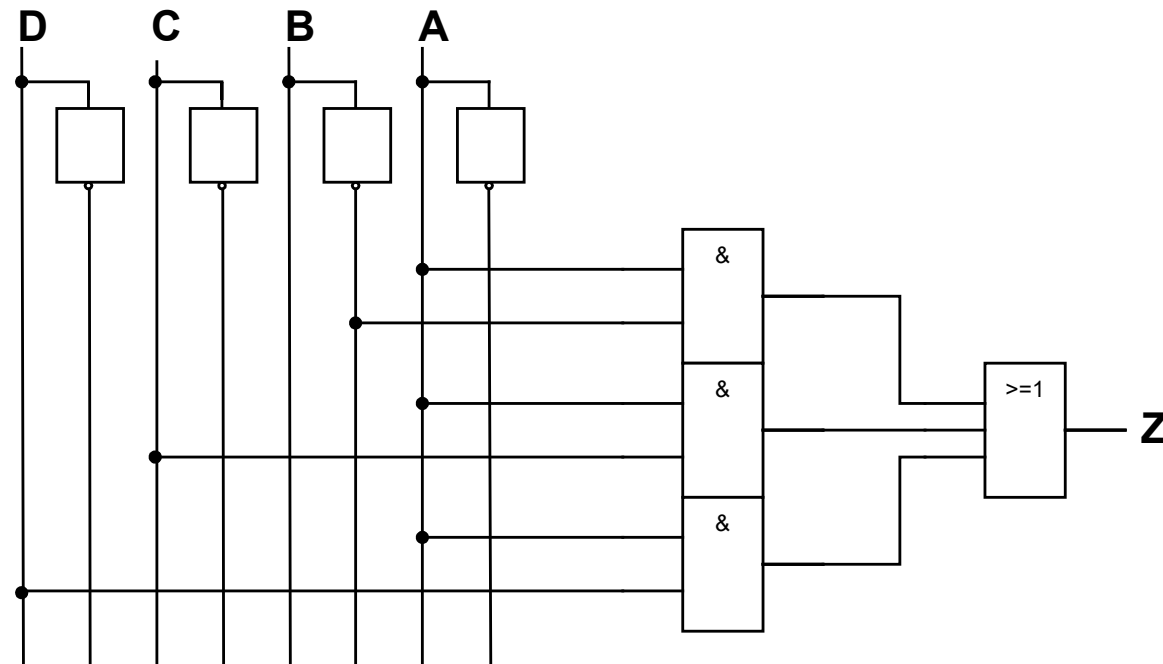


4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel V
 - Teil 3

$$Z = DA + CA + \overline{B}A$$

→ $\min(F_K) = \min(N_E + N_G) = 9 + 4 = 13$

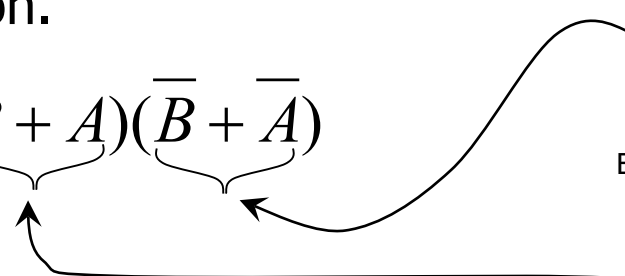
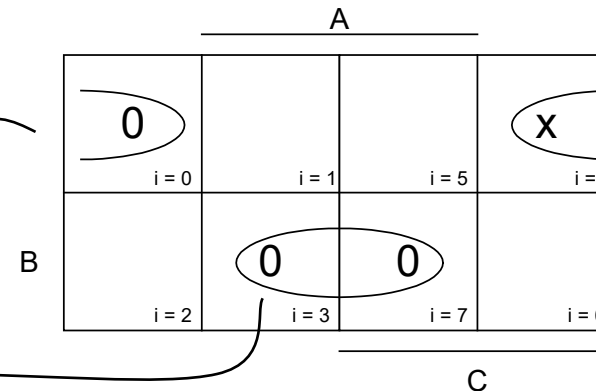


4.1.1.1 Minimierungsvorschriften nach Karnaugh

- Beispiel VI
 - Anstelle der Einsen werden nun die Nullen berücksichtigt. Zusätzlich soll davon ausgegangen werden, dass ein Ausgangswert ein Don't-Care-Wert ist. Es handelt sich somit um eine unvollständig definierte Funktion.

C	B	A	Z
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	X
1	0	1	1
1	1	0	1
1	1	1	0

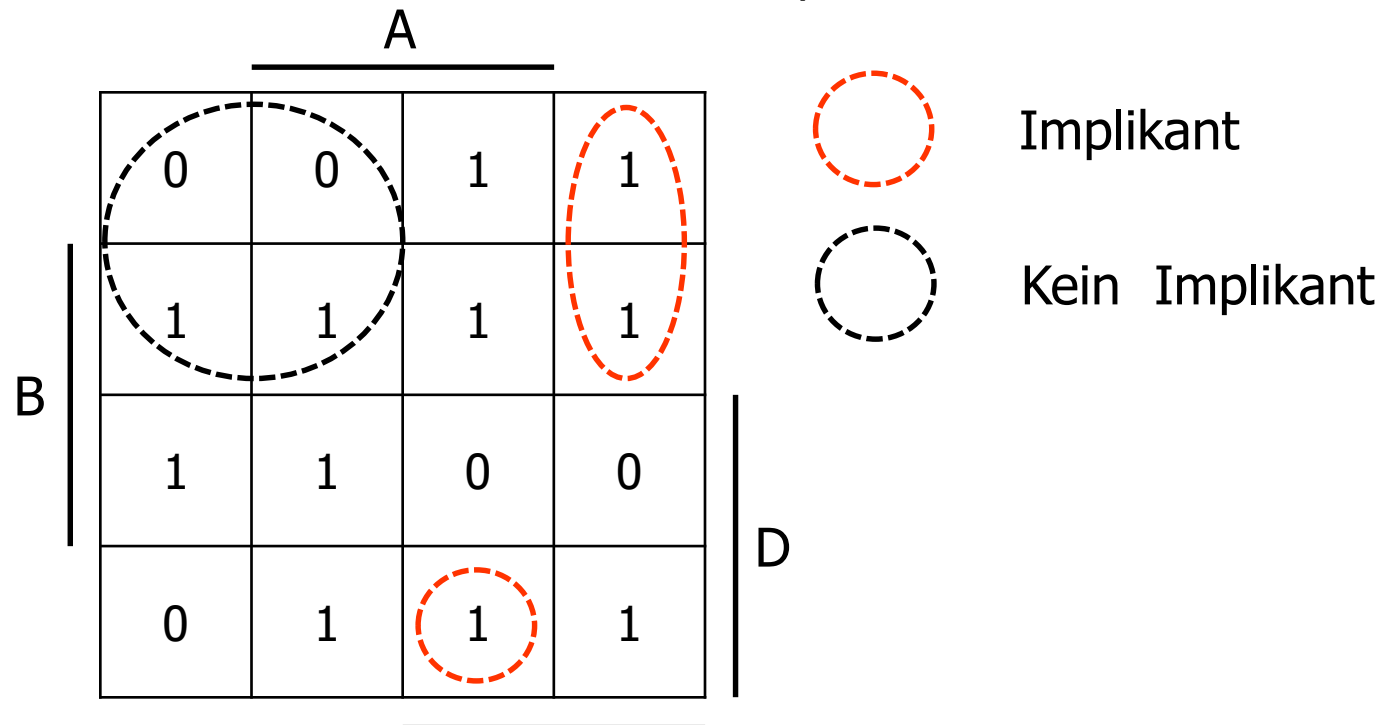
$$Z = (B + A)(\overline{B} + \overline{A})$$



4.1.1.2 Systematische Minimierung

■ Definition 4.2 (Implikant)

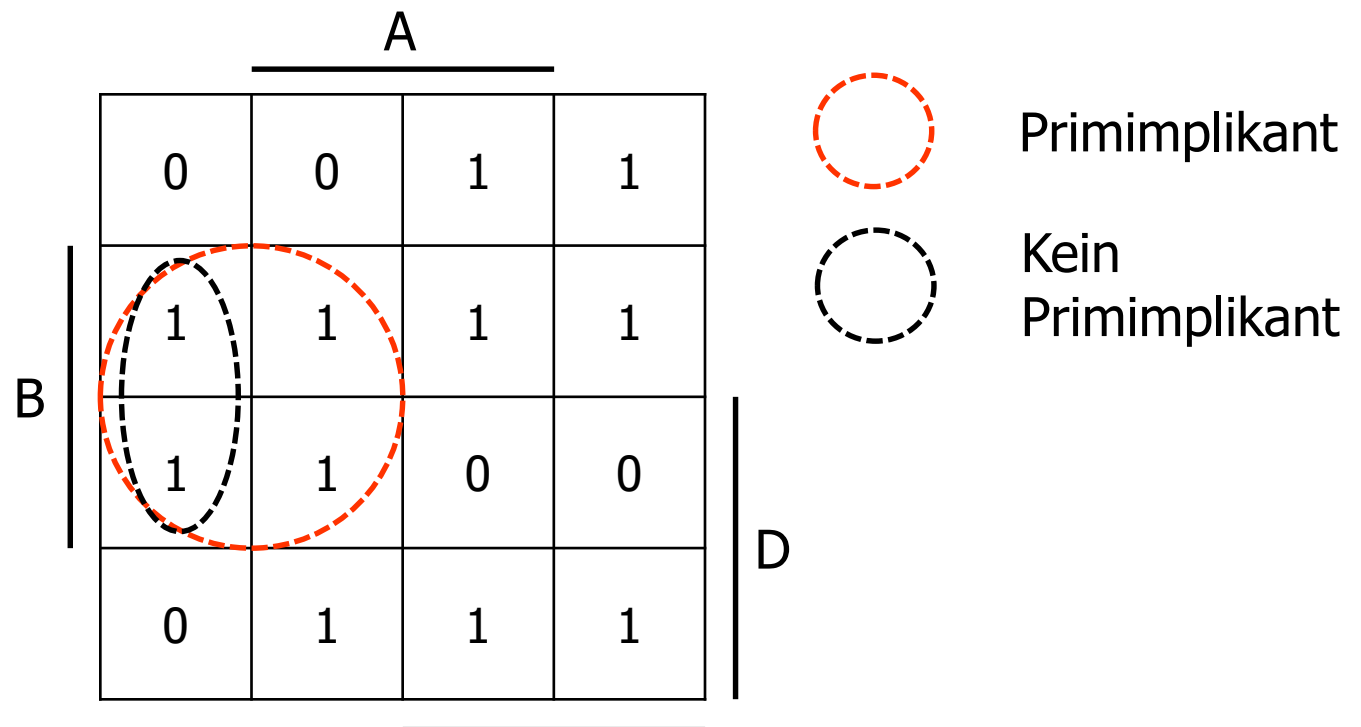
- Ein Implikant (I) einer Booleschen Funktion ist ein Produktterm (Konjunktion), der mindestens eine oder maximal alle Einsstellen einer Funktion f überdeckt. Damit ist ein Minterm ein Implikant.



4.1.1.2 Systematische Minimierung

■ Definition 4.3 (Primimplikant)

- Ein Primimplikant (PI) einer Booleschen Funktion ist ein Produktterm (Konjunktion), der selbst ein Implikant ist und für den kein anderer Implikant existiert, der ihn ganz oder teilweise überdeckt. Es besitzt damit mindestens eine Einsstelle mehr als alle anderen Implikanten der Funktion f .



4.1.1.2 Systematische Minimierung

- Beispiel

		A			
		0	0	1	1
		1	1	1	1
		1	1	0	0
		0	1	1	1
				C	
B					D

- Blakesche Normalform (kanonisch)

$$f_{PI} = \overline{D}C + \overline{C}B + \overline{D}B + C\overline{B} + D\overline{B}A + D\overline{C}A$$

4.1.1.2 Systematische Minimierung

■ Beispiel

- Folgende PI werden in jedem Fall benötigt (willkürlich):

A

	0	0	1	1
B	1	1	1	1
	1	1	0	0
	0	1	1	1

C

D

$$Z = \overline{D}C + C\overline{B} + \overline{C}B + D\overline{C}A$$

- Die disjunktive Minimalform (DMF) setzt sich aus einer Anzahl (Kern) N_K von Primimplikanten zusammen, die kleiner oder gleich der Anzahl N_{PI} aller vorkommenden Primimplikanten einer Booleschen Funktion f ist.

$$N_K \leq N_{PI}$$

- Die Anzahl aller Primimplikanten wird durch die Blakesche Normalform festgelegt.

4.1.1.2 Systematische Minimierung

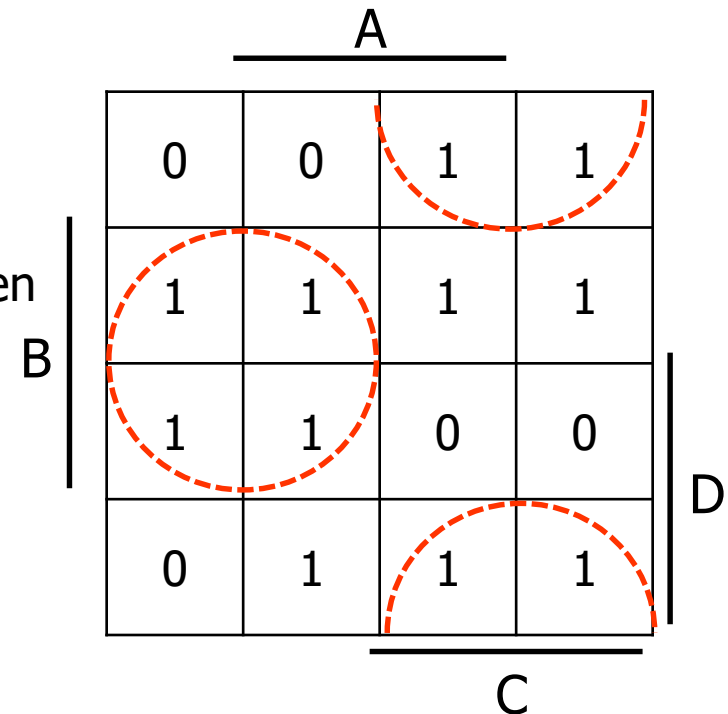
- Im nächsten Schritt muss geklärt werden, *welche* Primimplikanten einer Blakeschen Normalform zur Bildung einer DMF benötigt werden und nach welchen Kriterien diese ausgewählt werden können. Die Kostenfunktion nach Definition 4.1 steht dabei im Mittelpunkt der Überlegungen. Hierzu wird die folgende Definition benötigt.

- **Definition 4.4** (Kernprimimplikant)

- Ein Kernprimimplikant (KPI) stellt einen Primimplikanten dar, der von **keinem anderen Primimplikanten überdeckt** werden kann. Der KPI besitzt mindestens eine Einsstelle, die auch durch die Zusammenfassung aller anderen Primimplikanten sonst nicht dargestellt wird. Ein KPI wird auch als **wesentlicher Primimplikant** bezeichnet.

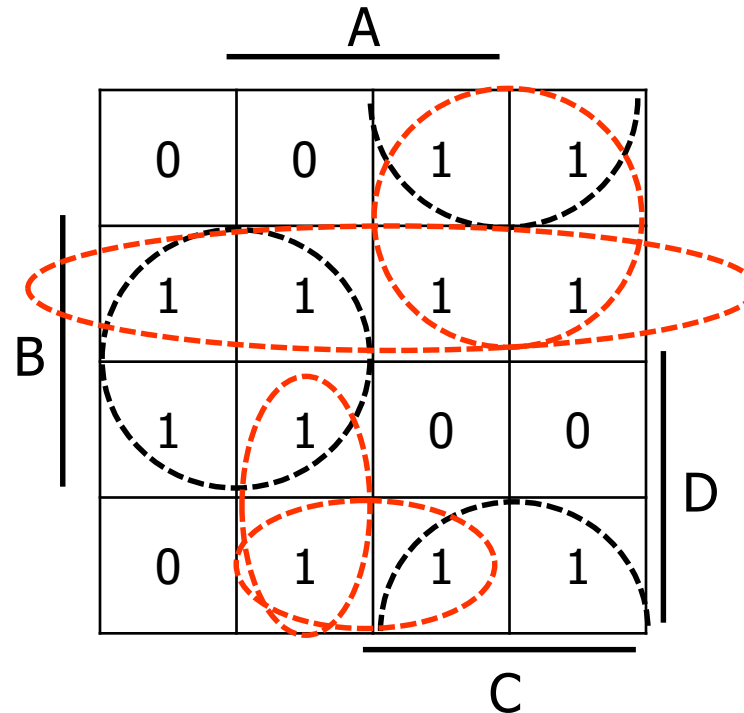
Stichworte:

Kostenfunktion, maximale Ausdehnung



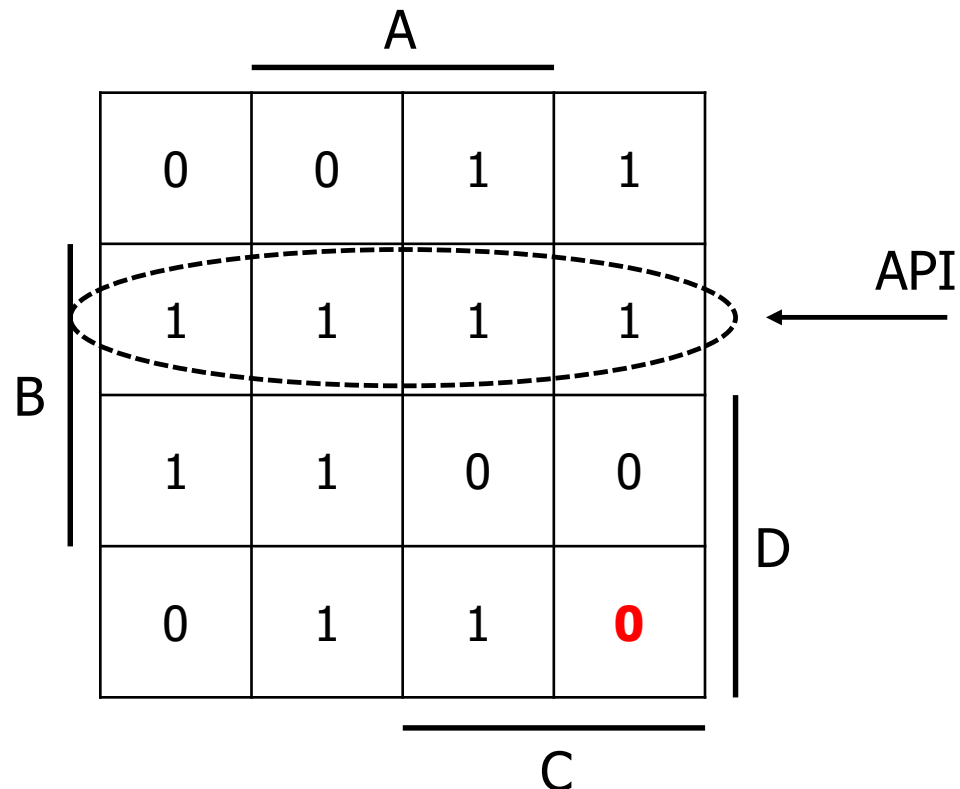
4.1.1.2 Systematische Minimierung

- **Definition 4.5** (relativ absorbierbarer Primimplikant)
 - Ein relativ absorbierbarer Primimplikant (RPI) besitzt mindestens eine Einsstelle, die nicht durch einen KPI überdeckt wird. Sie wird aber von einem oder mehreren RPIs überdeckt.



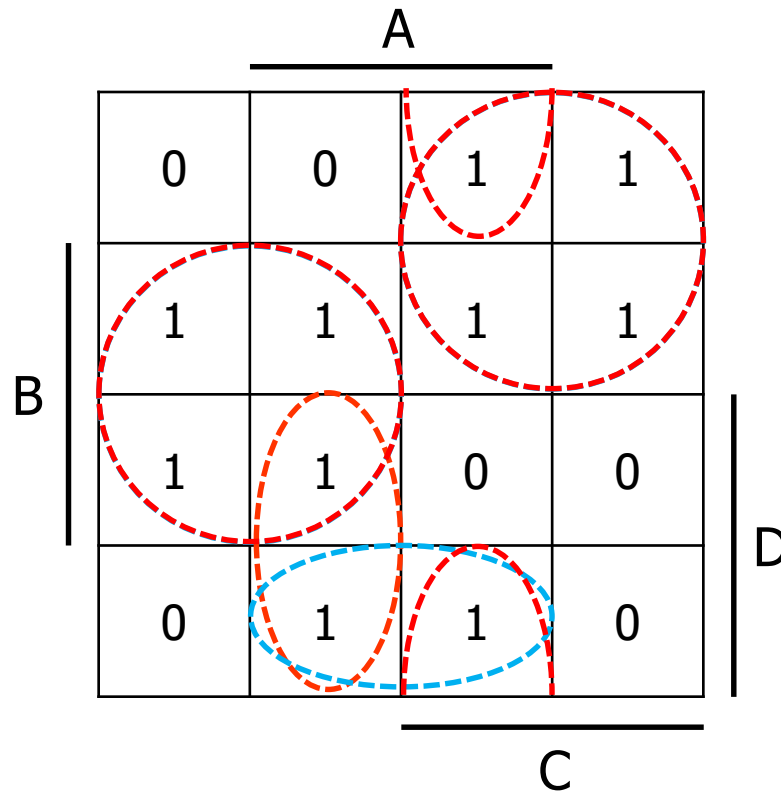
4.1.1.2 Systematische Minimierung

- **Definition 4.6** (absorbierbarer Primimplikant)
 - Ein absorbierbarer Primimplikant (API) besitzt keine Einsstelle, die nicht bereits durch einen Kernprimimplikanten überdeckt wurde.



4.1.1.2 Systematische Minimierung

■ Beispiel



Varianten:

$$f_1 = \overline{D}\overline{C} + \overline{C}\overline{B} + D\overline{B}A$$

$$F_{K_1} = 14$$

$$f_2 = \overline{D}\overline{C} + \overline{C}\overline{B} + D\overline{C}A + C\overline{B}A$$

$$F_{K_2} = 19$$

4.1.1.2 Systematische Minimierung

- Zusammenfassung

Die disjunktive Minimalform (DMF) setzt sich aus einer Anzahl N_K **Kernprimimplikanten** und einer geeignet auszuwählenden Anzahl N_R **relativ absorbierbarer Primimplikanten** zusammen.

$$N_K + N_R \leq N_{PI}$$

Erfüllen mehrere DNFs die genannten Bedingungen, so ist die DNF mit den geringsten Kosten als DMF auszuwählen. Eine Normalform, für die kein Produktterm weggelassen werden darf, nennt sich **nicht redundant**.

4.2 Algorithmische Verfahren

- Der K-Plan ist bis zu etwa 5 bis 6 Variablen geeignet. Darüber hinaus haben sich algorithmische Verfahren etabliert.
- Das klassische Verfahren nach [Quine-McCluskey](#) eignet sich etwa bis zu 20 Variablen, danach wird auch dieses Verfahren unübersichtlich. Das liegt zum einen an der Größe der abzuarbeitenden Tabellen, zu anderen steigt die Anzahl der Primimplikanten rapide mit der Anzahl der Eingangsvariablen an.
- Das Finden der KPIs und RPIs mit einer minimalen Überdeckung (*Minimal Cover*) wird dann unverhältnismäßig schwierig.
- Damit quasi-minimale DMF gefunden werden, ist man zu heuristischen Verfahren – z. B. Presto (ca. 1981) oder [Espresso](#) (ca. ab 1984, verschiedene Versionen, ständige Anpassungen, heute faktisch der Standard) – übergegangen. Sie liefern im Mittel optimale Lösungen – selbst bei Eingangsvariablenmengen von $N > 1000$.

4.2.1 Quine-McCluskey-Verfahren

- Das Verfahren basiert ebenfalls auf den Absorptionsregeln. Es wird wiederum eine Wahrheitstabelle als Beispiel genutzt:

i	D	C	B	A	Z
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	0

4.2.1 Quine-McCluskey-Verfahren

	2	3	4	5	6	7	9	10	11	12	13
PI0							x		x		
PI1							x				x
PI2	x	x			x	x					
PI3	x	x						⊗	x		
PI4			x	x	x	x					
PI5			x	x						⊗	x

Lösung: $Z = D\bar{B}A + \bar{C}B + \bar{D}C + C\bar{B}$

4.2.2 Algorithmische Verfahren

- Man versucht durch geschickte Erkennung der Primimplikanten, die gleichzeitige Überdeckung unterschiedlicher Minterme zu reduzieren. Dabei kann der Fall eintreten, dass nicht das absolute Minimum einer Kostenfunktion gefunden wird, sondern ein **relatives Minimum**.
- Da dieses jedoch den schaltungstechnischen Aufwand bereits erheblich senkt, kann ein derartiger Algorithmus **praxisgerecht** angewendet werden.
- Das Vorgehen ist bei den Algorithmen prinzipiell gleich: Zunächst wird versucht, alle Primimplikanten zu finden, um dann deren minimale Überdeckung zu bestimmen. Es bleibt der prinzipielle Nachteil erhalten, dass auch diese Werkzeuge nur für zweistufige Normalformen geeignet sind.

4.2.2 Algorithmische Verfahren

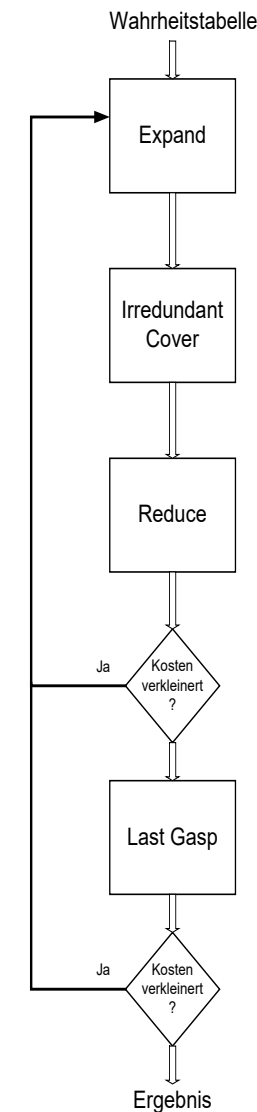
- Neben seinem Vorgänger *Presto* wird zumeist *Espresso* verwendet. Espresso ist in vielen kommerziell erhältlichen Werkzeugen implementiert.
- Die heutigen Bezeichnungen lauten *Espresso II* oder auch *WinEspresso*.
- Die Programme sind frei verfügbar und können aus dem Internet herunter geladen werden.

4.2.2 Algorithmische Verfahren

- Espresso
- 1. Schritt: Expandieren (*Expand*)
 - Espresso expandiert alle Implikanten auf Maximalgröße. Implikanten, die von expandierten Varianten überdeckt werden, schließt Espresso im Vorhinein aus. Es wird immer eine Funktion selbst und deren Komplement verwendet, um zu prüfen, ob eine Expansion tatsächlich die Überdeckung derselben vergrößert.
- 2. Schritt: Nicht redundante Überdeckung (*Irredundant Cover*)
 - Eine nicht redundante Überdeckung wird von den expandierten Implikanten extrahiert. Das Verfahren entspricht der Generierung der Primimplikanten-Tabelle nach QMC. Versucht möglichst frühzeitig KPIs zu entdecken, um diese dann abzuspeichern, da sie in jedem Fall notwendig sind.
- 3. Schritt: Reduktion (*Reduce*)
 - Die vorliegende Lösung ist bereits akzeptabel, jedoch kann häufig eine Verbesserung vorgenommen werden. Oft ergibt eine andere Überdeckung einen Ausdruck mit weniger Termen oder einer geringeren Anzahl von Literalen. Espresso reduziert die PIs auf eine minimale Größe, die gerade noch sicherstellt, dass alle Einstellen (Nullstellen) der Booleschen Funktion überdeckt werden.

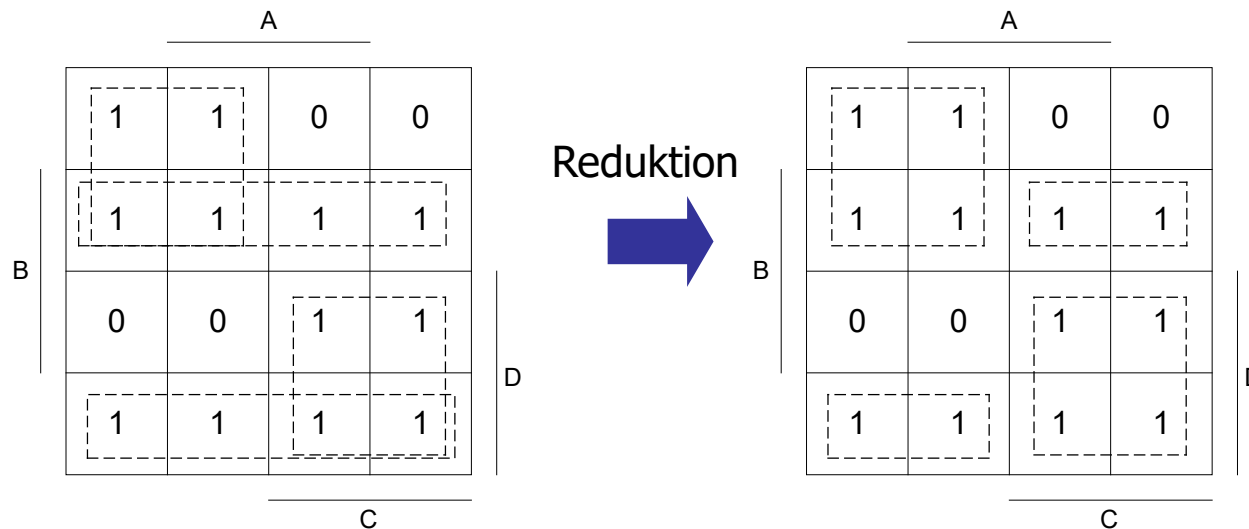
4.2.2 Algorithmische Verfahren

- 4. Schritt: Wiederholung (*Repeat*)
 - Die Reduktion nach Schritt 3 erzeugt in der Regel eine nicht prime Überdeckung. Man wiederholt die Schritte 1, 2 und 3 so lange, bis alternative PIs gefunden werden. Diese werden darauf geprüft, ob die neue Variante kostengünstiger ist als die Vorherige.
- 5. Schritt: Letzte Prüfung (*Last Gasp*, wörtlich übersetzt: *letzter Atemzug*)
 - Dieser Schritt soll sicherstellen, dass kein einzelner PI zur Überdeckung derart hinzugefügt werden kann, so dass sich danach beide eliminieren lassen.



4.2.2 Algorithmische Verfahren

- Prinzipielle Funktionsweise des Algorithmus'
 - Annahme die Schritte 1 und 2 sind bereits ausgeführt.



Die Anzahl der Literale ist
gestiegen, deshalb ...

4.2.2 Algorithmische Verfahren

- ... wird ein Expand durchgeführt.

